

Comparisons for reference_wrapper

Document #: P2944R3
Date: 2024-03-20
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 Ambiguity Issues](#)

[2.2 Non-boolean comparisons](#)

[2.3 Constraints vs Mandates](#)

[3 Proposal](#)

[3.1 Feature-test macro](#)

[4 References](#)

§ 1 Revision History

Since [[P2944R2](#)], wording fixes.

Since [[P2944R1](#)], added section on [ambiguity](#) and updated wording accordingly.

Since [[P2944R0](#)], fixed the wording

§ 2 Introduction

Typically in libraries, wrapper types are comparable when their underlying types are comparable. `tuple<T>` is equality comparable when `T` is. `optional<T>` is equality comparable when `T` is. `variant<T>` is equality comparable when `T` is.

But `reference_wrapper<T>` is a peculiar type in this respect. It looks like this:

```
template<class T> class reference_wrapper {
public:
    // types
    using type = T;

    // [refwrap.const], constructors
    template<class U>
        constexpr reference_wrapper(U&&) noexcept(see below);
    constexpr reference_wrapper(const reference_wrapper& x) noexcept;
```

```

// [refwrap.assign], assignment
constexpr reference_wrapper& operator=(const reference_wrapper& x)
    noexcept;

// [refwrap.access], access
constexpr operator T&() const noexcept;
constexpr T& get() const noexcept;

// [refwrap.invoke], invocation
template<class... ArgTypes>
constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...)
    const
    noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);
};

```

When `T` is not equality comparable, it is not surprising that `reference_wrapper<T>` is not equality comparable. But what about when `T` is equality comparable? There are no comparison operators here, but nevertheless the answer is... maybe?

Because `reference_wrapper<T>` is implicitly convertible to `T&` and `T` is an associated type of `reference_wrapper<T>`, `T`'s equality operator (if it exists) might be viable candidate. But it depends on exactly what `T` is and how the equality operator is defined. Given a type `T` and an object `t` such that `t == t` is valid, let's consider the validity of the expressions `ref(t) == ref(t)` and `ref(t) == t` for various possible types `T`:

<code>T</code>	<code>ref(t) == ref(t)</code>	<code>ref(t) == t</code>
builtins	✓	✓
class or class template with member <code>==</code>	✗	✓ (since C++20)
class with non-member or hidden friend <code>==</code>	✓	✓
class template with hidden friend <code>==</code>	✓	✓
class template with non-member, template <code>==</code>	✗	✗
<code>std::string_view</code>	✗	✓

That's a weird table!

Basically, if `T` is equality comparable, then `std::reference_wrapper<T>` is... sometimes... depending on how `T`'s comparisons are defined. `std::reference_wrapper<int>` is equality comparable, but `std::reference_wrapper<std::string>` is not. Nor is `std::reference_wrapper<std::string_view>` but you can nevertheless compare a `std::reference_wrapper<std::string_view>` to a `std::string_view`.

So, first and foremost: sense, this table makes none.

Second, there are specific use-cases to want `std::reference_wrapper<T>` to be normally equality comparable, and those use-cases are the same reason what `std::reference_wrapper<T>` exists to begin with: deciding when to capture a value by copy or by reference.

Consider wanting to have a convenient shorthand for a predicate to check for equality against a value. This is something that shows up in lots of libraries (e.g. Björn Fahlner's [lift](#) or Conor Hoekstra's [blackbird](#)), and looks something like this:

```
inline constexpr auto equals = [](auto&& value){
    return [value=FWD(value)](auto&& e){ return value == e; };
};
```

Which allows the nice-looking:

```
if (std::ranges::any_of(v, equals(0))) {
    // ...
}
```

But this implementation always copies (or moves) the value into the lambda. For larger types, this is wasteful. But we don't want to either unconditionally capture by reference (which sometimes leads to dangling) or write a parallel hierarchy of reference-capturing function objects (which is lots of code duplication and makes the library just worse).

This is *exactly* the problem that `std::reference_wrapper<T>` solves for the standard library: if I want to capture something by reference into `std::bind` or `std::thread` or anything else, I pass the value as `std::ref(v)`. Otherwise, I pass `v`. We should be able to use the exact same solution here, without having to change the definition of `equals`:

```
if (std::ranges::any_of(v, equals(std::ref(target)))) {
    // ...
}
```

And this works! Just... only for some types, seemingly randomly. The goal of this proposal is for it to just always work.

§ 2.1 Ambiguity Issues

In the original revision of the paper, the proposal was simply to add this equality operator:

```
template<class T> class reference_wrapper {
    friend constexpr bool operator==(reference_wrapper, reference_wrapper);
};
```

But this turns out to be insufficient. It's enough for `reference_wrapper<T>` to become comparable for all cases, but that's not exactly all we need. Consider:

```
auto check(int i, std::reference_wrapper<int> r) -> bool {
    return i == r;
}
```

This comparison is valid today, per the table earlier: we convert `r` to `int` through its `operator int&()` and use the builtin comparison. But now we're adding a new candidate, which is also valid: we can convert `i` to `reference_wrapper<int>`. These two candidates are ambiguous. The same is true for many other similar comparisons.

In order to ensure that we catch all the interesting cases, we can build up all the comparisons that we want to check. For non-const T:

```
template <class T>
concept ref_equality_comparable = requires (T a, T const ca, Ref<T> r, Ref<T
    const> cr) {
    // the usual T is equality-comparable with itself
    a == a;
    a == ca;
    ca == ca;

    // Ref<T> is equality-comparable with itself
    r == r;
    r == cr;
    cr == cr;

    // T and Ref<T> are equality-comparable
    a == r;
    a == cr;
    ca == r;
    ca == cr;
};
```

We don't need to check both directions of comparison anymore, but we do need to check const and non-const comparisons - which means T and T `const` for the objects and Ref<T> and Ref<T `const`> for our reference wrapper. We need to be careful to check both because of the case I just showed earlier - `int == reference_wrapper<int>` would be ambiguous with the rules laid out in R0 and R1 of this paper, but `int const == reference_wrapper<int>` actually would be fine (because `int const&` is not convertible to `reference_wrapper<int>`, so we only have one viable candidate).

That concept fails for every type with the R0/R1 proposal. To disambiguate, we need to add an extra comparison to handle the `T == Ref<T>` case:

```
template<class T> class reference_wrapper {
    friend constexpr bool operator==(reference_wrapper, reference_wrapper);
    friend constexpr bool operator==(reference_wrapper, T const&);
};
```

That gets us a lot closer, but it still isn't sufficient. Actually only one single expression now fails: the `r == cr` (Ref<T> == Ref<T `const`>) check, which fails for all T. The previous ambiguity is annoying, but this one particularly so since we just need a dedicated comparison operator *just* for this case. Which we can add:

```
template<class T> class reference_wrapper {
    friend constexpr bool operator==(reference_wrapper, reference_wrapper);
    friend constexpr bool operator==(reference_wrapper, T const&);
    friend constexpr bool operator==(reference_wrapper, reference_wrapper<T
        const>); // only for non-const T
};
```

And that, now, passes [all the tests](#).

§ 2.2 Non-boolean comparisons

Another question that came up with in the LEWG telecon was how this proposal interacts with non-boolean comparison operators. For instance:

```
void f(std::valarray<int> v) {  
    // this is a valid expression today, whose type is not bool, but rather  
    // something convertible to std::valarray<bool>  
    v == v;  
}
```

Now, `std::valarray<T>`'s comparison operators are specified as non-member function templates, so any comparison using `std::reference_wrapper<std::valarray<T>>` doesn't work today. But let's make our own version of this type that's more friendly (or hostile, depending on your perspective) to this paper and consider:

```
template <typename T>  
struct ValArray {  
    friend auto operator==(ValArray const&, ValArray const&) -> ValArray<bool>  
    {  
        return {};  
    }  
};  
  
void f(ValArray<int> v) {  
    // this is valid and has type ValArray<bool>  
    v == v;  
  
    // this is also valid today and has the same type  
    std::ref(v) == std::ref(v);  
}
```

Now, does anybody write such code? Who knows. If we [constrain](#) the comparisons of `std::reference_wrapper<T>` (and also the other standard library types), then this code will continue to work fine anyway - since the comparisons would be constrained away by types like `ValArray<T>` not satisfying `equality_comparable`. This paper would not be adding any new candidates to the candidate set, so no behavior changes.

But, as always, there is an edge case.

1. there is a type `T`, whose comparisons return a type like `int`
2. and those comparisons are written in such a way that comparison `T` to `std::reference_wrapper<T>` works (see table above)
3. and users are relying on such comparisons to actually return `int`

Then the comparisons to `std::reference_wrapper<T>` will instead start returning `bool`. That is:

```
struct ComparesAsInt {  
    friend auto operator==(ComparesAsInt, ComparesAsInt) -> int;  
};
```

```

auto f(std::reference_wrapper<ComparesAsInt> a,
      std::reference_wrapper<ComparesAsInt> b) {
    // today: compiles and returns int
    // proposed: compiles and returns bool
    return a == b;
}

```

Here, the added comparison operators would be valid, and wouldn't constrain away, since `std::equality_comparable` is based on *boolean-testable* which only requires convertibility to `bool` (and some other nice behavior), which `int` does satisfy. And those added comparison operators would be better matches than the existing ones, so they would win.

This would be the only case where any behavior would change.

§ 2.3 Constraints vs Mandates

Surprisingly, the status quo today is that for standard library types `std::pair`, `std::tuple`, etc., the spaceship operator is constrained (by way of *synth-three-way-result*<T>) but the equality operators and existing relational operators actually are *Mandated* instead. There does not seem to be a particularly good reason for this. It kind of just happened - the relational comparisons became constrained by way of my [P1614R2], and the equality ones just weren't touched. It would make a lot more sense to have all of them constrained, so that `std::equality_comparable<std::tuple<T>>` wasn't just `true` for all T (well, except `void` and incomplete types).

This paper proposes as a drive-by to also make all the comparison operators *Constrained* instead of *Mandated*.

§ 3 Proposal

Add `==` and `<=>` to `std::reference_wrapper<T>` so that `std::reference_wrapper<T>` is always comparable when T is, regardless of how T's comparisons are defined.

Change 22.10.6.1 [refwrap.general]:

```

template<class T> class reference_wrapper {
public:
    // types
    using type = T;

    // [refwrap.const], constructors
    template<class U>
        constexpr reference_wrapper(U&&) noexcept(see below);
    constexpr reference_wrapper(const reference_wrapper& x) noexcept;

    // [refwrap.assign], assignment
    constexpr reference_wrapper& operator=(const reference_wrapper& x)
        noexcept;

    // [refwrap.access], access
    constexpr operator T& () const noexcept;
}

```

```

constexpr T& get() const noexcept;

// [refwrap.invoke], invocation
template<class... ArgTypes>
constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...)
    const
    noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);

+ // [refwrap.comparisons], comparisons
+ friend constexpr bool operator==(reference_wrapper, reference_wrapper);
+ friend constexpr bool operator==(reference_wrapper, const T&);
+ friend constexpr bool operator==(reference_wrapper,
    reference_wrapper<const T>);

+ friend constexpr synth-three-way-result<T> operator<==>
    (reference_wrapper, reference_wrapper);
+ friend constexpr synth-three-way-result<T> operator<==>
    (reference_wrapper, const T&);
+ friend constexpr synth-three-way-result<T> operator<==>
    (reference_wrapper, reference_wrapper<const T>);
};

```

...

- 3 The template parameter `T` of `reference_wrapper` may be an incomplete type.

[*Note 1*: Using the comparison operators described in subclause [refwrap.comparisons] with `T` being an incomplete type can lead to an ill-formed program with no diagnostic required ([temp.point], [temp.constr.atomic]) — *end note*]

Add a new clause, [refwrap.comparisons], after 22.10.6.5 [refwrap.invoke]:

- ```

friend constexpr bool operator==(reference_wrapper x, reference_wrapper y);

```
- 1 *Constraints*: The expression `x.get() == y.get()` is well-formed and its result is convertible to `bool`.
  - 2 *Returns*: `x.get() == y.get()`.
- ```

friend constexpr bool operator==(reference_wrapper x, const T& y);

```
- 3 *Constraints*: The expression `x.get() == y` is well-formed and its result is convertible to `bool`.
 - 4 *Returns*: `x.get() == y`.
- ```

friend constexpr bool operator==(reference_wrapper x,
 reference_wrapper<const T> y);

```
- 5 *Constraints*: `is_const_v<T>` is `false` and the expression `x.get() == y.get()` is well-formed and its result is convertible to `bool`.
  - 6 *Returns*: `x.get() == y.get()`.

```

friend constexpr synth-three-way-result<T> operator<==>(reference_wrapper x,
 reference_wrapper y);

7 Returns: synth-three-way(x.get(), y.get()).

friend constexpr synth-three-way-result<T> operator<==>(reference_wrapper x,
 const T& y);

8 Returns: synth-three-way(x.get(), y).

friend constexpr synth-three-way-result<T> operator<==>(reference_wrapper x,
 reference_wrapper<const T> y);

9 Constraints: is_const_v<T> is false.

10 Returns: synth-three-way(x.get(), y.get()).

```

And then additional drive-by changes for existing library types as follows.

In 22.3.3 [\[pairs.spec\]](#)/1:

1 ~~Preconditions~~ Constraints: `x.first == y.first` and `x.second == y.second` are valid expressions and each ~~Each~~ of `decltype(x.first == y.first)` and `decltype(x.second == y.second)` models *boolean-testable*.

In 22.4.9 [\[tuple.rel\]](#)/2:

2 ~~Mandates~~ Constraints: For all `i`, where `0 <= i < sizeof...(TTypes)`, `get<i>(t) == get<i>(u)` is a valid expression and `decltype(get<i>(t) == get<i>(u))` models *boolean-testable*. `sizeof...(TTypes)` equals `tuple_size_v<UTuple>`.

3 ~~Preconditions: For all `i`, `decltype(get<i>(t) == get<i>(u))` models *boolean-testable*.~~

In 22.5.6 [\[optional.relops\]](#), change all the *Mandates* to *Constraints*:

1 ~~Mandates~~ Constraints: The expression `*x == *y` is well-formed and its result is convertible to `bool`.

4 ~~Mandates~~ Constraints: The expression `*x != *y` is well-formed and its result is convertible to `bool`.

7 ~~Mandates~~ Constraints: The expression `*x < *y` is well-formed and its result is convertible to `bool`.

10 ~~Mandates~~ Constraints: The expression `*x > *y` is well-formed and its result is convertible to `bool`.

13 ~~Mandates~~ Constraints: The expression `*x <= *y` is well-formed and its result is convertible to `bool`.



- 16 ~~Mandates~~ *Constraints*: The expression  $*x \geq *y$  is well-formed and its result is convertible to `bool`.

In 22.5.8 [\[optional.comp.with.t\]](#), change all the *Mandates* to *Constraints*:

- 1 ~~Mandates~~ *Constraints*: The expression  $*x == v$  is well-formed and its result is convertible to `bool`.
- 3 ~~Mandates~~ *Constraints*: The expression  $v == *x$  is well-formed and its result is convertible to `bool`.
- 5 ~~Mandates~~ *Constraints*: The expression  $*x != v$  is well-formed and its result is convertible to `bool`.
- 7 ~~Mandates~~ *Constraints*: The expression  $v != *x$  is well-formed and its result is convertible to `bool`.
- 9 ~~Mandates~~ *Constraints*: The expression  $*x < v$  is well-formed and its result is convertible to `bool`.
- 11 ~~Mandates~~ *Constraints*: The expression  $v < *x$  is well-formed and its result is convertible to `bool`.
- 13 ~~Mandates~~ *Constraints*: The expression  $*x > v$  is well-formed and its result is convertible to `bool`.
- 15 ~~Mandates~~ *Constraints*: The expression  $v > *x$  is well-formed and its result is convertible to `bool`.
- 17 ~~Mandates~~ *Constraints*: The expression  $*x \leq v$  is well-formed and its result is convertible to `bool`.
- 19 ~~Mandates~~ *Constraints*: The expression  $v \leq *x$  is well-formed and its result is convertible to `bool`.
- 21 ~~Mandates~~ *Constraints*: The expression  $*x \geq v$  is well-formed and its result is convertible to `bool`.
- 23 ~~Mandates~~ *Constraints*: The expression  $v \geq *x$  is well-formed and its result is convertible to `bool`.

In 22.6.6 [\[variant.relops\]](#), change all the *Mandates* to *Constraints*:

- 1 ~~Mandates~~ *Constraints*:  $\text{get}\langle i \rangle(v) == \text{get}\langle i \rangle(w)$  is a valid expression that is convertible to `bool`, for all  $i$ .
- 3 ~~Mandates~~ *Constraints*:  $\text{get}\langle i \rangle(v) != \text{get}\langle i \rangle(w)$  is a valid expression that is convertible to `bool`, for all  $i$ .

- 5 *Mandates Constraints*: `get<i>(v) < get<i>(w)` is a valid expression that is convertible to `bool`, for all `i`.
- 7 *Mandates Constraints*: `get<i>(v) > get<i>(w)` is a valid expression that is convertible to `bool`, for all `i`.
- 9 *Mandates Constraints*: `get<i>(v) <= get<i>(w)` is a valid expression that is convertible to `bool`, for all `i`.
- 11 *Mandates Constraints*: `get<i>(v) >= get<i>(w)` is a valid expression that is convertible to `bool`, for all `i`.

## § 3.1 Feature-test macro

We don't have a feature-test macro for `std::reference_wrapper<T>`, and there doesn't seem like a good one to bump for this, so let's add a new one to 17.3.2 [\[version.syn\]](#)

```
+ #define __cpp_lib_reference_wrapper 20XXXXL // freestanding, also in
 <functional>
```

Likewise, let's add a new one for the other standard library types described above:

```
+ #define __cpp_lib_constrained_equality 20XXXXL // freestanding, also in
 <utility>, <tuple>, <optional>, <variant>
```

## § 4 References

---

[P1614R2] Barry Revzin. 2019-07-28. The Mothership Has Landed: Adding `<=>` to the Library.  
<https://wg21.link/p1614r2>

[P2944R0] Barry Revzin. 2023-07-09. Comparisons for `reference_wrapper`.  
<https://wg21.link/p2944r0>

[P2944R1] Barry Revzin. 2023-08-17. Comparisons for `reference_wrapper`.  
<https://wg21.link/p2944r1>

[P2944R2] Barry Revzin. 2023-09-17. Comparisons for `reference_wrapper`.  
<https://wg21.link/p2944r2>